

Data Pipeline

A **data pipeline** is a series of steps that **move data** from one system to another. The goal is to **collect, process, and store** data in a reliable, scalable, and efficient way.

Key Components of a Data Pipeline:

1. Sources:

- Where data comes from: databases (SQL/NoSQL), APIs, logs, IoT devices, files (CSV, JSON), etc.

2. Ingestion:

- Capturing data from sources.
- Could be batch (periodic loads) or streaming (real-time).

3. Processing / Transformation:

- Cleaning, filtering, aggregating, joining, enriching, etc.
- May include ETL or ELT processes (more on that below).

4. Storage / Destination:

- Data warehouse, data lake, or operational systems.

5. Orchestration & Scheduling:

- Tools like **Apache Airflow**, **Luigi**, or **Dagster** help coordinate and automate workflows.

6. Monitoring & Logging:

- Ensures reliability, detects failures, and audits data movement.

Challenges in Building Data Pipelines

- **Data Quality:** Missing or inconsistent data
- **Latency:** Speed of data movement
- **Error Handling:** Retry, alerting, dead-letter queues
- **Data Governance:** Who has access, data lineage, privacy (GDPR, HIPAA)
- **Scalability:** Handling growth in data volume

Why Use Data Pipelines?

1. **Automation:** Removes manual data processing.
2. **Scalability:** Handles large volumes of data efficiently.
3. **Consistency:** Ensures data quality and integrity.
4. **Speed:** Enables real-time or near real-time processing.
5. **Reusability:** Modular and reusable components.

Comparing Tools Based on Your Needs

Let's compare popular tools in four main categories:

◆ A. Ingestion

Tool	Best For	Notes
Fivetran	Fully managed ingestion	Easy to use, expensive at scale
Kafka	Real-time ingestion	Complex setup, highly scalable
Airbyte	Open-source ELT	Flexible connectors, free tier

Tool	Best For	Notes
AWS Glue	AWS-native ingestion & ETL	Serverless, great with S3

◆ B. Transformation

Tool	Best For	Notes
dbt	SQL transformations	Modular, testable, version-controlled
PySpark	Large-scale data processing	Python + Spark, distributed
SQL	Simple transformations	Easy for analysts
Pandas	Local or small-scale	Good for prototypes, not scalable

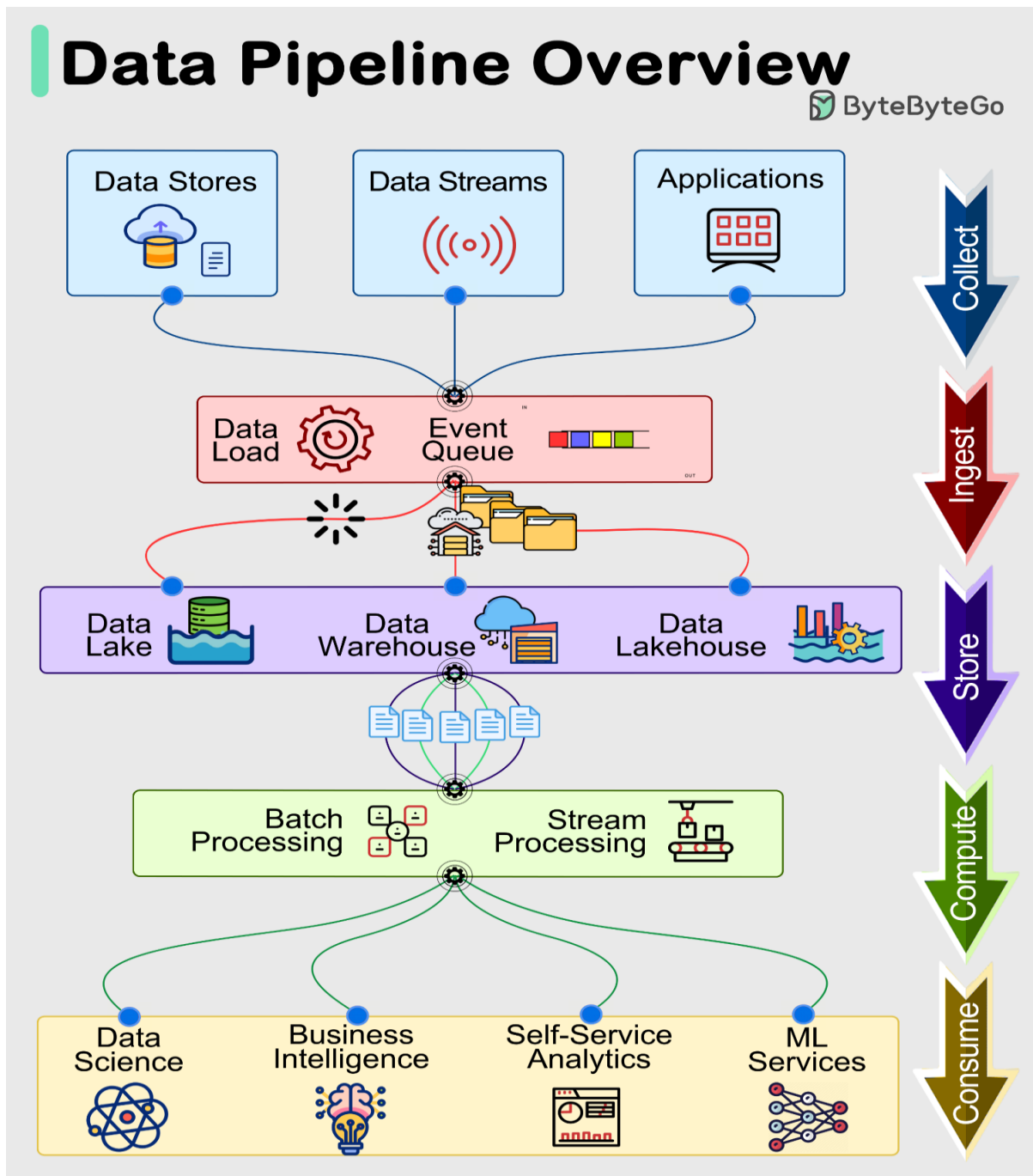
◆ C. Orchestration

Tool	Best For	Notes
Airflow	Complex pipelines	Popular, scalable, Python-based
Prefect	Modern alternative to Airflow	Easier setup, dynamic workflows
Dagster	Data-aware orchestration	Great observability and testing
Luigi	Lightweight batch pipelines	Simpler but less flexible

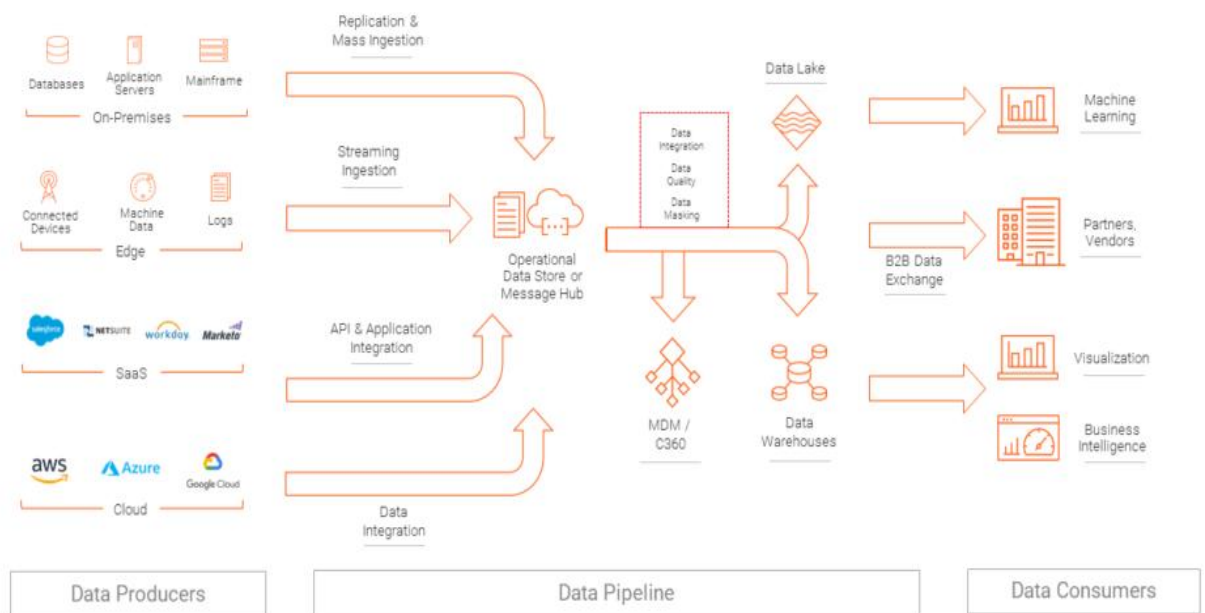
◆ D. Storage

Tool	Type	Notes
Snowflake Data Warehouse		Great performance, SQL-based

Tool	Type	Notes
BigQuery	Serverless DW	Great for GCP, usage-based pricing
S3	Data Lake	Object storage for raw data
Redshift	Warehouse	Good for AWS users



Data Pipeline Patterns



What is ETL?

ETL is a **type of data pipeline** specifically focused on preparing data for analytics and reporting.

✓ 1. Extract

Definition: Retrieve data from one or more sources.

- **Examples of sources:**

- Relational DBs (MySQL, PostgreSQL)
- APIs (REST, GraphQL)
- Flat files (CSV, XML, JSON)
- Cloud storage (S3, GCS)
- Message queues (Kafka, RabbitMQ)

- **Challenges:**

- Schema changes
- API rate limits
- Inconsistent formats

✓ 2. Transform

Definition: Clean, structure, and enrich data.

- **Examples of transformations:**

- Removing duplicates
- Standardizing date formats
- Aggregating metrics (e.g., daily sales totals)
- Joining tables
- Deriving new columns

- **Tools:** SQL, Pandas, PySpark, dbt (data build tool)

- **Business logic** often lives here.

✓ 3. Load

Definition: Write the transformed data into the final destination.

- **Destinations:**

- **Data warehouse** (BigQuery, Snowflake, Redshift)
- **Data lake** (S3, Azure Data Lake)
- BI tools or dashboards (Tableau, Power BI)

Real-World Example

Imagine an **e-commerce company**:

1. **Extract:**

- Orders from MySQL
- Customer data from Salesforce API
- Web events from logs in S3

2. **Transform:**

- Clean missing customer fields
- Join orders with customer info
- Calculate revenue per product

3. **Load:**

- Load into Snowflake for reporting
- Power BI dashboards use this data

ETL vs. Data Pipeline

Feature	ETL	Data Pipeline
Purpose	Prepare data for analytics	Move and process data from one point to another
Includes	Extract, transform, load	ETL/ELT + other steps (e.g., streaming, ML)

Feature	ETL	Data Pipeline
Data Type	Typically batch	Batch + real-time
Tools	ETL-specific tools	Broader ecosystem (e.g., Kafka, Airflow)

Designing a Data Pipeline Architecture

Let's design a **modern data pipeline architecture** for a **typical business use case** — e.g., an **e-commerce company** collecting data from multiple sources and loading it into a **data warehouse** for analytics.

Use Case: E-commerce Analytics Platform

Goals:

- Collect data from transactional databases, website activity logs, and CRM APIs.
- Clean and transform data.
- Load into a warehouse like Snowflake or BigQuery.
- Enable dashboarding and reporting via BI tools.

Architecture Overview:

```

+-----+
| Web / App |
+-----+-----+
|

```

(User Events)

|

+-----v-----+

| Kafka / Kinesis | ← Real-time ingestion

+-----+-----+

|

v

+-----+-----+

| Spark Streaming | ← Real-time processing

+-----+

|

v

+----+----+

| S3 / | ← Raw event storage (Data Lake)

| Data |

| Lake |

+----+----+

||

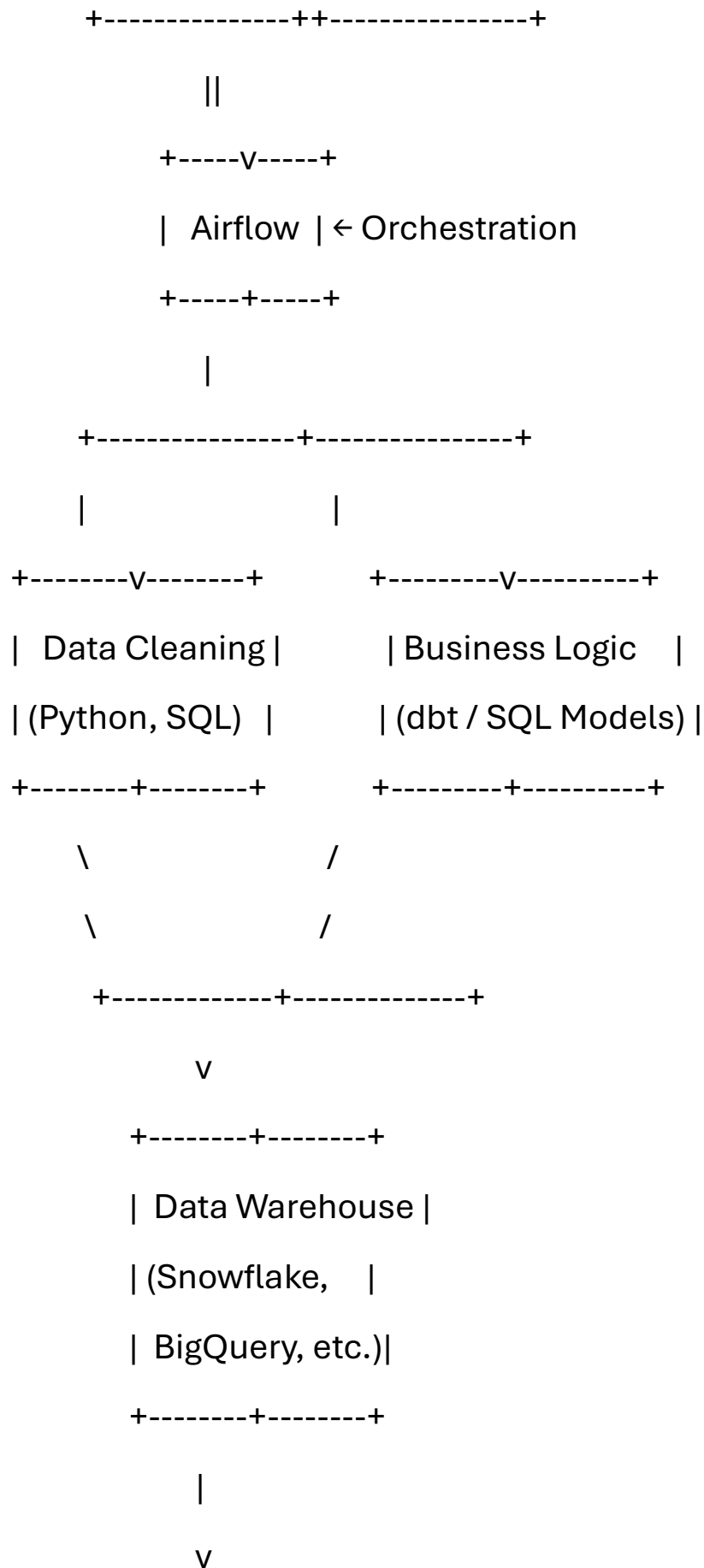
+-----+ || +-----+

| MySQL | || | Salesforce API |

| (Orders) | || | (CRM Data) |

+-----+ || +-----+

| || |



+-----+-----+

| BI Tools (e.g., |

| Looker, PowerBI)|

+-----+

✓ Key Considerations:

- **Streaming + Batch:** Combine Kafka (stream) and Airflow (batch).
- **Data Lake** for raw historical data.
- **Warehouse** for fast analytics.
- **dbt** for modular, testable transformations.
- **Monitoring:** Set alerts for job failures (e.g., via Airflow or Prometheus + Grafana).